# COM203 - Computer Architecture

## Lab # 3

### Mars



Dr. Mohammadmahdi Ariannejad

mohammadmahdi.ariannejad@xmu.edu.my

# Outline

- System Calls
- PRINTING AN INTEGER
- PRINTING A FLOAT



# System Calls



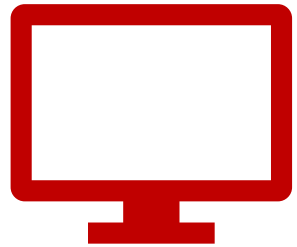
## Introduction

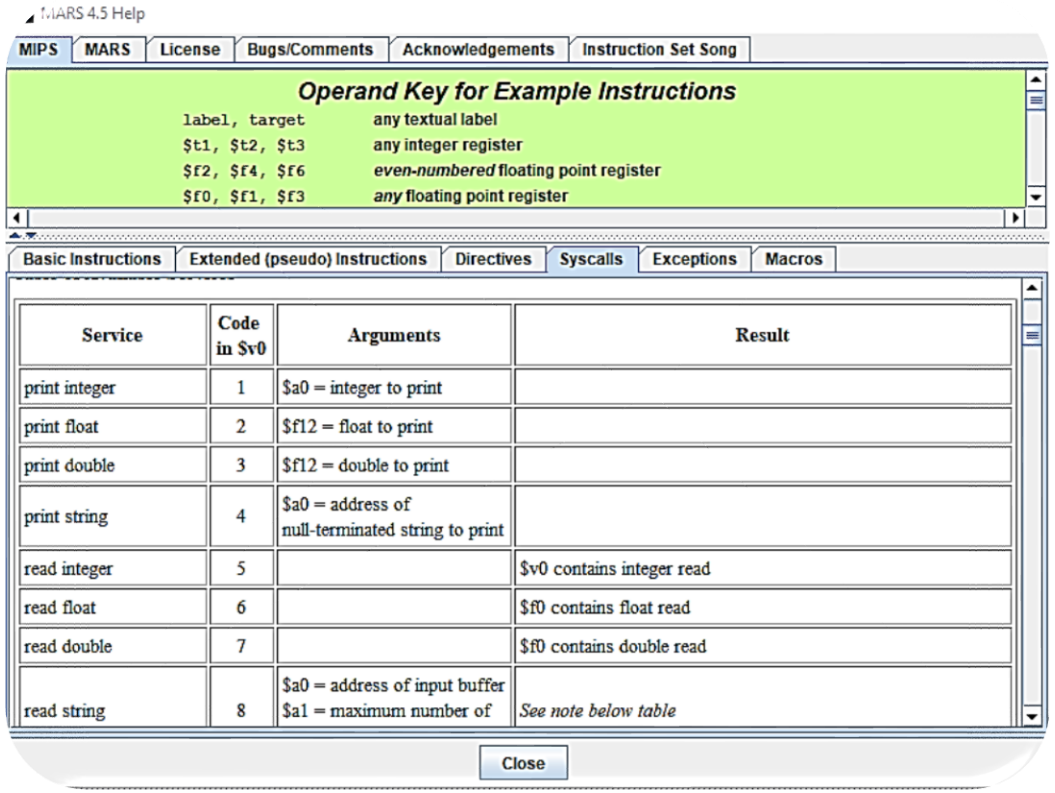
- A number of system services, mainly for **input and output**, are available for use by your MIPS program.

- MIPS register contents are **not affected** by a system call, except for **result registers**.

**Table:** System services.

| Service | System Call Code | Arguments | Result |
|---|---|---|---|
| print_int | 1 | $a0 = integer | |
| print_float | 2 | $f12 = float | |
| print_double | 3 | $f12 = double | |
| print_string | 4 | $a0 = string | |
| read_int | 5 | | integer (in $v0) |
| read_float | 6 | | float (in $f0) |
| read_double | 7 | | double (in $f0) |
| read_string | 8 | $a0 = buffer, $a1 = length | |
| sbrk | 9 | $a0 = amount | address (in $v0) |
| exit | 10 | | |
| print_character | 11 | $a0 = character | |
| read_character | 12 | | character (in $v0) |
| open | 13 | $a0 = filename, | file descriptor (in $v0) |
| | | $a1 = flags, $a2 = mode | |
| read | 14 | $a0 = file descriptor, | bytes read (in $v0) |
| | | $a1 = buffer, $a2 = count | |
| write | 15 | $a0 = file descriptor, | bytes written (in $v0) |
| | | $a1 = buffer, $a2 = count | |
| close | 16 | $a0 = file descriptor | 0 (in $v0) |
| exit2 | 17 | $a0 = value | |





# How to use SYSCALL system services

- **Step 1.** Load the service number in **register $v0**.
- **Step 2.** Load argument values, if any, in $a0, $a1, $a2, or $f12 as specified.
- **Step 3.** Issue the **SYSCALL** instruction.
- **Step 4.** Retrieve return values, if any, from **result registers** as specified.

# Termination of Program:

- System call code **'10'** is used to terminate the program. To terminate the executing program, the value '10' is required to be copied to register **$v0** and then the instruction syscall will be executed to perform the **termination task**.

```
li      $v0, 10
syscall
```

# INTEGER INPUT:

- System call code **'5'** is used to get an integer input. After calling the syscall, user has to input the number. This input is stored in the register **$v0**. Which has to be **moved to some other register** before proceeding forward.

```
li        $v0, 5
syscall

move $t0, $v0
```

# INTEGER OUTPUT:

- System call code **'1'** is used to display an **integer value** on the screen. The value must be stored in the register **$a0** hence before calling syscall, we have to move the required value to the register **$a0**. In the below example, we want to display the value of register **$t0**, so we have **moved the contents** of register **$t0** to the register **$a0 .**

```
move $a0, $t0
li   $v0, 5
```

# syscall

- Move is data movement instruction. The number of operands required for the move instruction in **two**. One of them is source while the other is destination. The general syntax of move instruction is:

***move Rdest, Rsrc***

Where *Rdest* is the **destination register** while *Rsrc* is the **source register** and we are moving value from *Rsrc* to *Rdest*.

# PRINTING AN INTEGER

# &

# PRINTING A FLOAT

# PRINTING AN INTEGER

Code:

```
.data
        Myage: .word  88    #This is a word or integer
.text
        # Print an integer to the screen.
        li    $v0, 1
        lw  $a0, Myage      #putting the value in a0
        syscall
        li $v0, 10          #Exit command
        syscall
```

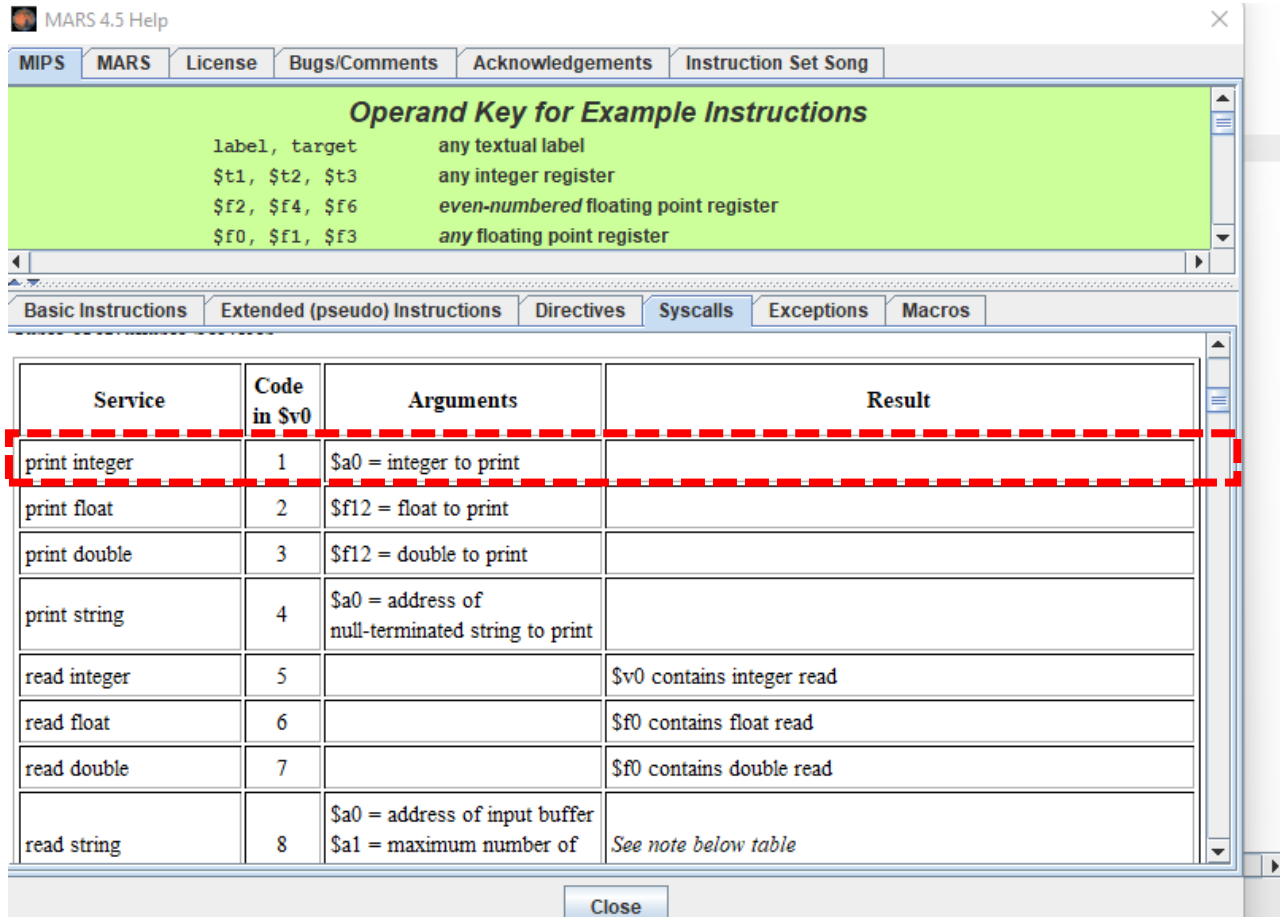
# PRINTING A FLOAT

Code:

```
.data
    PI: .float 3.141592

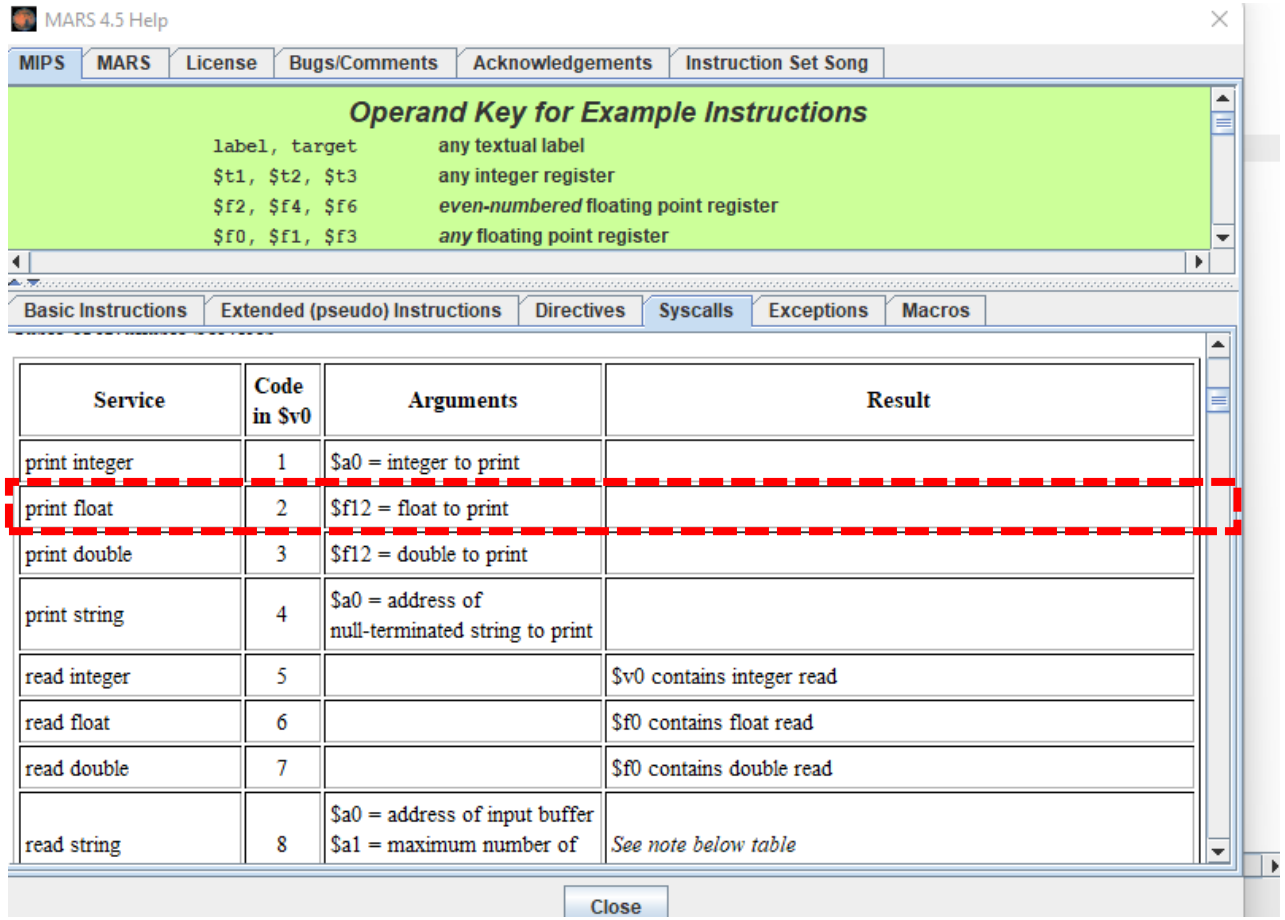
.text
    li    $v0, 2
    lwc1  $f12, PI      #from random access memory , always need to be f12 (coprocessor 1) for float numbers
    syscall
    li $v0, 10                  #Exit command
    syscall
```

# Print Integer



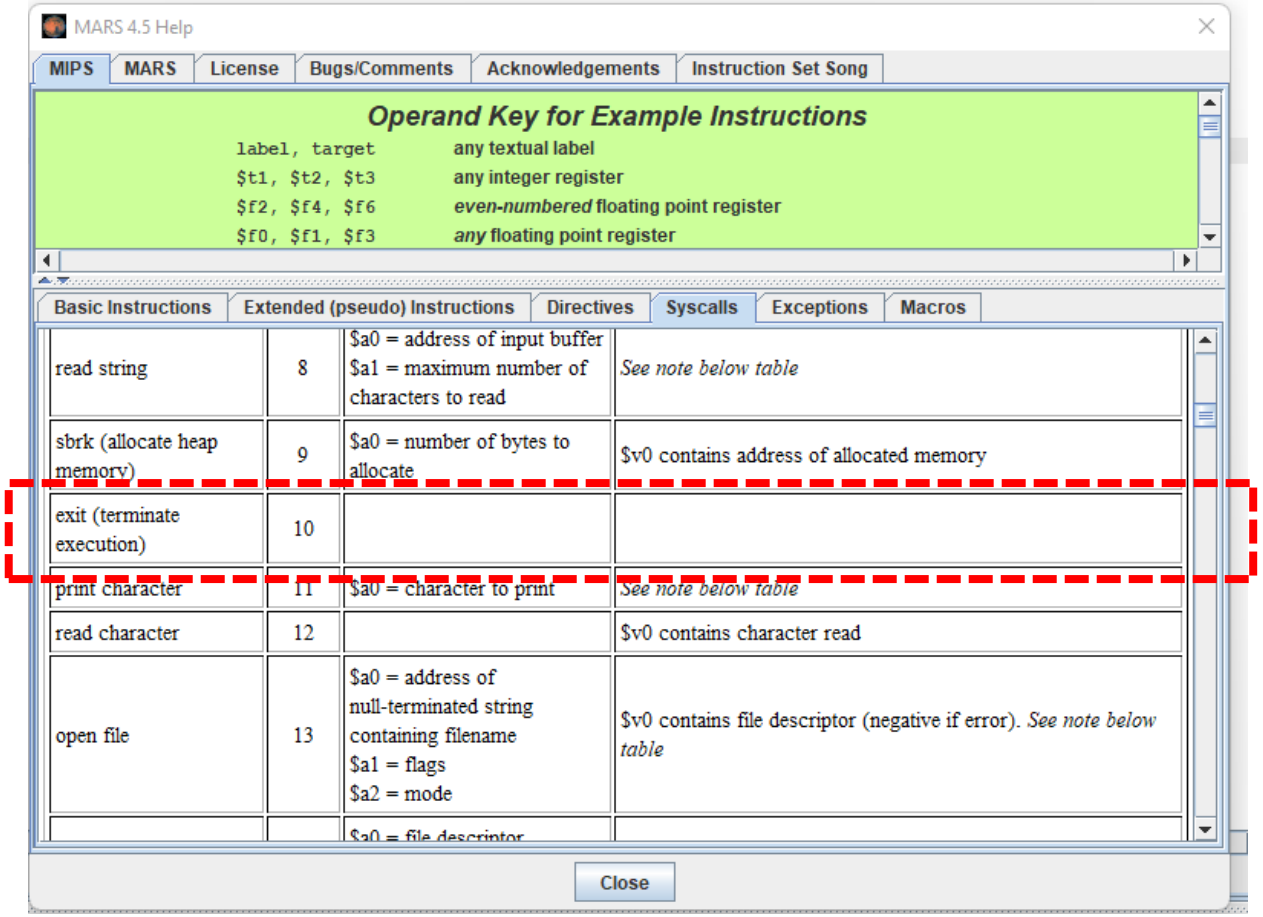

| Service | Code in $v0 | Arguments | Result |
|---|---|---|---|
| print integer | 1 | $a0 = integer to print | |
| print float | 2 | $f12 = float to print | |
| print double | 3 | $f12 = double to print | |
| print string | 4 | $a0 = address of null-terminated string to print | |
| read integer | 5 | | $v0 contains integer read |
| read float | 6 | | $f0 contains float read |
| read double | 7 | | $f0 contains double read |
| read string | 8 | $a0 = address of input buffer<br>$a1 = maximum number of | *See note below table* |

# Print FLOAT




| Service | Code in $v0 | Arguments | Result |
|---|---|---|---|
| print integer | 1 | $a0 = integer to print | |
| print float | 2 | $f12 = float to print | |
| print double | 3 | $f12 = double to print | |
| print string | 4 | $a0 = address of null-terminated string to print | |
| read integer | 5 | | $v0 contains integer read |
| read float | 6 | | $f0 contains float read |
| read double | 7 | | $f0 contains double read |
| read string | 8 | $a0 = address of input buffer<br>$a1 = maximum number of | *See note below table* |

# EXIT


MARS 4.5 Help
MIPS
MARS
License
Bugs/Comments
Acknowledgements
Instruction Set Song
Operand Key for Example Instructions
label, target any textual label
$t1, $t2, $t3 any integer register
$f2, $f4, $f6 even-numbered floating point register
$f0, $f1, $f3 any floating point register
Basic Instructions
Extended (pseudo) Instructions
Directives
Syscalls
Exceptions
Macros
read string 8 $a0 = address of input buffer $a1 = maximum number of characters to read See note below table
sbrk (allocate heap memory) 9 $a0 = number of bytes to allocate $v0 contains address of allocated memory
exit (terminate execution) 10
print character 11 $a0 = character to print See note below table
read character 12 $v0 contains character read
open file 13 $a0 = address of null-terminated string containing filename $a1 = flags $a2 = mode $v0 contains file descriptor (negative if error). See note below table
$a0 = file descriptor
Close

# End of LAB SESSION! ☺

- **Next week:** Printing a Double & Adding Integers

- **Questions?**

